

17. Bölüm

Ön İşlemci Yönergeleri

17.1 Ön İşlemci Yönergesi Nasıl Çalışır?

C dilinde **ön işlemci** (**preprocessor**) yönergeleri, derleyicinin bir parçası değildir, ancak derleme sürecinde ayrı bir adımdır. Ön işlemci, yalnızca bir metin değiştirme aracıdır ve gerçek derlemeden önce gerekli ön işlemleri yapmasını sağlar. Derleme süreci, Şekil 17.1’de gösterilmiştir. Kısaca derleme önce bul-değiştir mantığı ile kodumuzda değişiklikler yapar.

- ◇ Ön işleme bir C kodunun derlenmesi öncesindeki ilk adımdır.
- ◇ Kodu belirteçlere ayırma (**tokenization**) adımından önce gerçekleşir.
- ◇ Ön işlemcinin önemli işlevlerinden biri de programda kullanılan kütüphane işlevlerini içeren başlık dosyalarını koda dahil etmek için kullanılmasıdır.
- ◇ Ön işlemci ayrıca değişmezleri (**literal**) tanımlar ve makro kullanımını sağlar.

C dilindeki ön işlemci ifadelerine, **yönerge** (**directive**) denir. Programda ön işlemci bölümü her zaman C kodunun en üstünde görünür. Her ön işlemci ifadesi, kare (**hash**) (**#**) sembolüyle başlar. Aşağıda ön işlemci yönergelerinin listesi verilmiştir;

Yönerge	Açıklama
#define	Ön işlemci makrosunu değiştirmek ya da ilk defa tanımlamak için kullanılır.
#include	Bir başlık dosyasını diğer ile dahil etmek için kullanılır.
#undef	Tanımlanmış bir makroyu tanımsız hale getirmek için kullanılır.
#ifdef	Bir makro tanımlı ise doğru/true değerini döndürür.
#ifndef	Bir makro tanımlı değilse doğru/true değerini döndürür.
#if	Derleme zamanında bir durumun kontrolü için kullanılır.
#else	#if Yönergesinde alternatif durumu ifade eder.
#elif	#else ve #if yönergelerini tek bir şekilde ifade etmek için kullanılır.
#endif	Şart ön işlemcilerini (#if , #else , #elif) bitirir.
#error	stderr standart dosyasına hata mesajını yazar.
#pragma	Derleyiciye özel komutlar vermek için kullanılır.

Tablo 17.1: Ön İşlemci Yönergeleri

17.2 Ön İşlemci Makroları

#define yönergesi ile yeni bir makro tanımlanabileceği gibi tanımlanmış bir makro **#undef** ile ortadan kaldırabilir veya yenisini tanımlayabiliriz. Aşağıdaki örnekte **PI** adlı bir makro tanımlanmış ve 3.1415 reel sayısını vermektedir. Daha önce **PI** adlı bir makro tanımlanmış ise derleyici hata verir. Aşağıda buna bir örnek verilmiştir;

```
#include <stdio.h>
#define PI 3.1415 // Yeni bir PI makrosu tanımlanıyor. Daha önce PI adlı makro tanımlı ise hata
→ alırız.
#ifndef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI daha önceden makro olarak TANIMSIZ ise
#define ENFAZLAOGRECISAYISI 100 // Yeni bir ENFAZLAOGRECISAYISI makrosu tanımlanıyor
#endif
int main() {
    printf("PI:%f\n",PI); //PI:3.141500
    printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI); //ENFAZLAOGRECISAYISI:100
#ifdef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI daha önceden makro olarak TANIMLI ise
```

```
#undef ENFAZLAOGRECISAYISI // ENFAZLAOGRECISAYISI makrosunu kaldır!
#define ENFAZLAOGRECISAYISI 20 // Yeni bir ENFAZLAOGRECISAYISI makrosu tanımlanıyor
#endif
printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI); //ENFAZLAOGRECISAYISI:20
return 0;
}
```

Kod içerisinde değişmezleri (**literal**) tekrar tekrar yazmamanın bir yolu da **#define ön işlemci yönergesi** (**preprocessor directive**) ile **makro tanımlamaktır**. Ön işlemciler, kaynak kodları derlemeden önce derleyiciyi yönlendirerek kaynak kodlar üzerinde değişiklik yapmayı sağlar. Aşağıdaki tanımlanan **PI** makrosuyla derleyici, derlemeye geçmeden **PI** görülen yerleri **3.14** olarak değiştirmesi söylenir. Derleme bu ön işlemci işleminden sonra gerçekleşir. Makro kimlikleri sabit değişkenler gibi **büyük harfle kimliklendirilir**.

Bilgi

Talimatlar noktalı virgül karakteriyle biter. **Ancak, ön işlemci makroları sonunda noktalı virgül karakteri ile olmaz!**

```
/* Bu program, #define ön işlemci yönergesi örneğidir. */

#define PI 3.14 /* Bu makro, derleme öncesinde PI görülen yerlere 3.14 yazılmasını sağlar. */
int main() {
    int yariCap=3;
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0*PI*yariCap;
    return 0;
}
```

Ön işlemci yönergesini yerine getiren derleyici aynı kodu aşağıdaki şekle çevirerek derleme işlemine geçer. Ayrıca ön işlemci yönergeleri yerine getirilmeden önce bütün açıklamalar ve **beyaz boşluklar** (**white space**) kaynak koddan kaldırılır. Bu durumda ön işlemci yönergeleri sonrasında bir üstte verilen kaynak kodumuz aşağıdaki hale döner;

```
int main(){int yariCap=3;float daireninAlani=3.14*yariCap*yariCap;float
↪ daireninCevresi=2.0*3.14*yariCap;return 0;}
```

#define ön işlemci yönergeleri ile çok tekrarlanan değişmezler için makro tanımlanabildiği gibi aşağıdaki örnek verildiği şekilde kod tasnifi için de makro kullanılabilir;

```
/* Bu program, #define ön işlemci yönergesinin bir başka kullanım örneğidir. */
#define BEGIN int main() {
#define END }
#define PI 3.14
BEGIN
const char ILKHARF='A';
const float AVAGADROSAYISI=6.022e23;
float yarimRadyan=PI/2;
return 0;
END
```

17.3 Parametrelili Ön İşlemci Makroları

Ön işlemci yönergeleriyle tanımlanan makrolar derlemeye geçilmeden önce işlem görür. Dolayısıyla bu aşamada bazen parametrelerle işlem yapılması gerekebilir.

```
#define kare(x) ((x) * (x))
#define kup(x) ((x) * (x) * (x))
#define buyugu(x,y) ((x) > (y) ? (x) : (y))
```

Burada kullanılacak parametrelerde veri tipi tanımlanmaz. Dolayısıyla kullanılacağı yerlere buna dikkat edilerek parametrelili makro tanımlanmalıdır. Eğer makro birkaç satırdan oluşacak ise ters bölü (**back slash**) (****) karakteri kullanılır.

```
#include <stdio.h>

#define SAYIYAZ(num, str) { \
    /* 1.Satir */ \
```

```

printf("%d :", num);    \
/* 2.Satir */          \
printf(" %s Numara \n", str); \
/* Son satırda ter bölü olmaz! */ \
}

#define MY_MULTI_LINE_MACRO(arg1, arg2) do { \
/* Line 1 of the macro */          \
printf("Arg1: %d\n", arg1);        \
/* Line 2 of the macro */          \
printf("Arg2: %d\n", arg2);        \
/* Last line (no backslash) */     \
} while (0);

int main() {
    SAYIYAZ(4, "Dort")
    MY_MULTI_LINE_MACRO(1,2)
    return 0;
}

/*Program Çıktısı:
4 : Dort Numara
Arg1: 1
Arg2: 2
*/

```

Makrolarda birden fazla işlem yapmak için virgül işleci (**comma operator**) kullanılır. Özellikle geriye tek bir değer dönmesi gereken **inline** makrolarda sırayla işlem yürütmek için tercih edilir:

```

#include <stdio.h>

#define ISLEM_YAP(x) (x++, x * 2)

int main() {
    int sayi = 4;

    int sonuc = ISLEM_YAP(sayi);
    // 1. sayi 1 artırılır (sayi = 5 olur)
    // 2. 5 * 2 hesaplanır (10)
    // 3. Virgül operatörü en sağdaki sonucu (10) geriye döndürür

    printf("Güncel sayı: %d\n", sayi); // Güncel sayı: 5
    printf("Dönen sonuç: %d\n", sonuc); // Dönen sonuç: 10

    return 0;
}

```

17.4 Ön Tanımlı Ön İşlemci Makroları

Her C derleyicisi için standart olarak tanımlı ön işlemci yönergesi ile makrolar bulunmaktadır;

Makro	Açıklama
__DATE__	Mevcut tarihi "MMM DD YYYY" biçiminde metin (string) olarak tanımlıdır.
__TIME__	Mevcut saat "HH:MM:SS" biçiminde metin (string) olarak tanımlıdır.
__FILE__	Dosya adı biçiminde metin (string) olarak tanımlıdır.
__LINE__	Dosyadaki satır numarası tamsayı (integer) olarak tanımlıdır.
__STDC__	ANSI standardında derleme yapılıyorsa 1 olarak tanımlıdır.

Tablo 17.2: Standart Ön İşlemci Makroları

Standart makroları kullanan örnek program aşağıda verilmiştir;

```

1 #include <stdio.h>
2 int main() {
3     printf("File: %s\n", __FILE__ ); //File: main.c

```

```

4   printf("Date: %s\n", __DATE__ ); //Date: Mar 22 2025
5   printf("Time: %s\n", __TIME__ ); //Time: 15:32:08
6   printf("Line: %d\n", __LINE__ ); //Line: 6
7   printf("ANSI: %d\n", __STDC__ ); //ANSI: 1
8 }

```

DEBUG Makrosu da hazır tanımlanmış bir makrodur ve hata ayıklama (**debug**) modunda kod derlenmesi halinde **#if** yönergelerinde doğru/true değerini döndürür. Bunun için derleme sırasında derleyiciye **-DDEBUG** argümanı verilir. Buna aşağıda bir örnek verilmiştir;

```

#include <stdio.h>
// DEBUG modunu aktifleştirmek için tanımlıyoruz (Derleyici parametresiyle de verilebilir)
#define DEBUG

// PI önceden tanımlıysa çakışmayı önlemek için önce tanımsız yapıyoruz veya #ifndef kullanıyoruz
#ifndef PI
#define PI 3.1415
#endif

#ifndef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI tanımlı değilse
#define ENFAZLAOGRECISAYISI 100 // Yeni makro değeri atanır
#endif

int main() {
    printf("PI:%f\n", PI); // Çıktı: PI:3.141500
    #ifdef DEBUG
        /* DEBUG tanımlı olduğu için bu blok derlemeye dahil edilir */
        printf("ENFAZLAOGRECISAYISI:%d\n", ENFAZLAOGRECISAYISI); // Çıktı: ENFAZLAOGRECISAYISI:100
    #endif
    return 0;
}

```

17.5 Kullanıcı Tanımlı Başlık Dosyası

#include yönergesi ile; hazır olan başlık dosyalarını küçük (<) ve büyük (>) karakterleri arasında yazarak koda dahil ederiz. Hazır olmayan ve programcı tarafından hazırlanan başlık dosyalarını ise çift tırnak (") karakterleri arasında yazarak koda dahil ederiz. Aşağıda kullanıcı tanımlı bir başlık dosyası **Baslik.h** örneği verilmiştir;

```

#ifndef BASLIK_H
#define BASLIK_H
// BASLIK_H makrosu sadece tanımlanıyor. Yerine bir metin konulmuyor

#define PI 3.1415
#define ENFAZLAOGRECISAYISI 100
float* ogrenciNotlari() {
    static float notlar[ENFAZLAOGRECISAYISI];
    return notlar;
}
float ogrenciNotlarOrtalamasi(float* pNotlar, int pOgranciSayisi) {
    int sayac;
    float ortalama=0;
    for (sayac=0; sayac<pOgranciSayisi; sayac++) {
        ortalama+=pNotlar[sayac];
        #ifdef DEBUG
            printf("%d. Ogrencide Ortalama:%f\n", sayac, ortalama);
            // Buradaki kod hata ayıklama modunda derlendiğinde çalışır.
            // Bunun için derleyiciye -DDEBUG argümanı verilir
        #endif
    }
    return ortalama/sayac;
}
#endif

```

Başlık dosyasının bir koda birden fazla dahil (**include**) edilmesini önlemek için `_BASLIK_H_` makrosu, şartlı (**conditional**) olarak tanımlanmıştır. Bu başlık dosyası bir kod projesinde birden fazla dahil olması halinde, `_BASLIK_H_` tanımlanmaz ise, başlık içindeki değişken ve fonksiyonlar birden fazla aynı kimlikle tanımlanacağından derleme hatası alınacaktı.

Bilgi

Bir başlık (**header**) dosyasında bildirimi yapılan bir fonksiyon, C derleyicisi tarafından dahil edileceği kaynak dosyaların tamamında yalnızca bir kez tanımlanabilir. Buna **tek tanım kuralı** (**one definition rule**) adı verilir. Kısaca birden çok dosya için derleme söz konusu olduğunda birden çok bildirim olabilir ama bir adet fonksiyon tanımı olmalıdır.

Kendi hazırlamış olduğumuz `baslik.h` başlık dosyasını kodumuza dahil ettiğimizde artık başlık dosyamızdaki fonksiyon ve değişkenleri kullanabilir hale geliriz. Aynı klasörde/dizinde bulunacak şekilde bu başlık dosyasını kullanan `main.c` örnek programı aşağıda verilmiştir;

```
#include <stdio.h>
#include "baslik.h"
int main() {
    printf("PI=%f\n",PI);
    printf("En fazla öğrenci Sayısı=%d\n",ENFAZLAOGRECISAYISI);
    float* notlar=ogrenciNotlari();
    notlar[0]=100;
    notlar[1]=80;
    notlar[2]=60;
    float ortalama=ogrenciNotlarOrtalamasi(notlar,3);
    printf("%d öğrenci için Ortalama %f",3,ortalama);
}
```

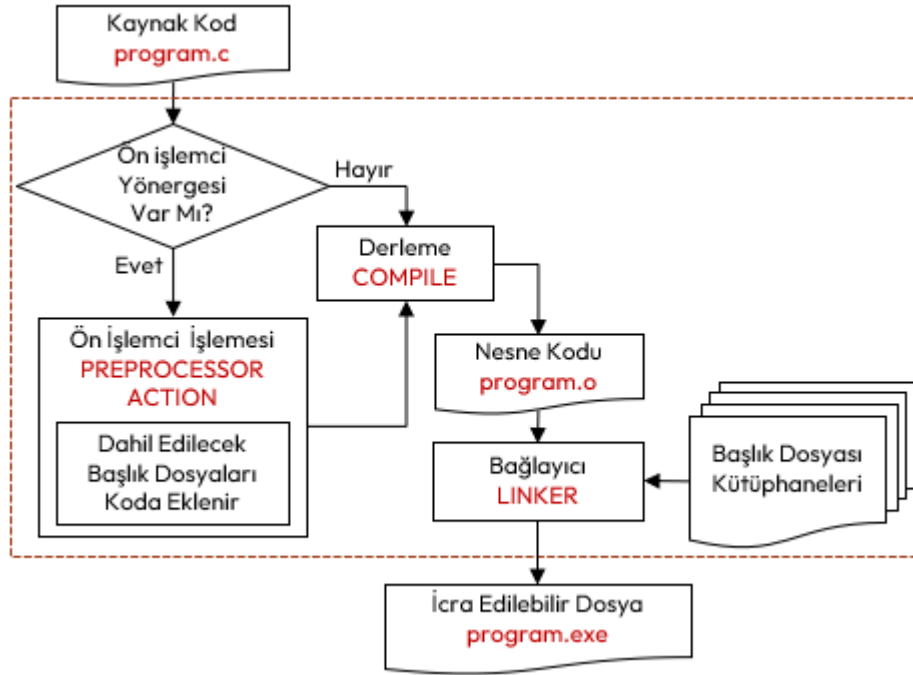
17.6 Derleme Sürecinin Detayı

C dilinde derleme sürecinin iki aşamadan oluştuğu daha önce anlatılmıştı. Aşağıda tüm süreç verilmiştir;

1. Birinci aşama:
 - (a) Kaynak koddaki tüm açıklamalar silinir.
 - (b) `#define`, `#if`, `#include`, gibi tüm yönergeler ile verilen işlemler yapılır.
 - (c) Bunlar arasında başlık dosyalarının koda dahil edilmesi de vardır.
 - (d) Bu duruma gelen kod, `gcc -E main.c` komutu dosya haline getirilebilir.
2. İkinci aşama:
 - (a) Kaynak kod montaj koda (assembly code) çevrilir.
 - (b) Bu duruma gelen kod, `gcc -S main.c` komutu dosya haline getirilebilir.
 - (c) Assembly kod derlenerek makine diline çevrilir ve amaç dosya (**object file**) oluşur.
 - (d) Bu duruma gelen kod, `gcc -c main.c` komutu ile dosya haline getirilebilir.
 - (e) Amaç dosya kodda belirtilen başlık dosyalarına uygun kütüphaneler (**library**) ile birbirine bağlayıcı (**linker**) ile bağlanır ve icra edilebilir dosya (**executable file**) elde edilir. Burada bahsedilen kütüphaneler her işletim sistemi için ayrı olarak oluşturulur ve derleyici kurulumunda bir klasörde tutulur.
 - (f) Bu aşamaya gelen derleme `gcc main.c -o main.exe` komutuyla icra edilebilir dosya oluşturulmaktadır.

17.7 Düşün ve Çöz

- ◊ **Düşün:** `#define` ile tanımlanan makroların, normal fonksiyonlara göre avantajları ve riskleri (yan etkileri) nelerdir?
- ◊ **Düşün:** Başlık dosyalarında kullanılan `#ifndef`, `#define`, `#endif` başlık güvenliği (**header guard**) bloklarının mükerrer dahil etmeyi nasıl önlediğini açıklayınız.
- ◊ **Düşün:** Koşullu derleme (`#ifdef DEBUG`) kullanarak, programın sadece geliştirme aşamasında ekrana teknik loglar basmasını nasıl sağlarsınız?
- ◊ **Çöz:** Bir sayının karesini alan bir makro (`#define KARE(x)`) tanımlayınız. `KARE(3+2)` ifadesinin sonucunu makronun nasıl açıldığını düşünerek analiz ediniz.
- ◊ **Çöz:** İki sayının küçüğünü bulan bir fonksiyonel makro (`#define MIN(a,b)`) yazınız ve makro **yan etkilerini** (**side effects**) önlemek için neden parantez kullanılması gerektiğini gösteriniz.
- ◊ **Çöz:** C'nin ön tanımlı makrolarını (`__DATE__`, `__TIME__`, `__FILE__`) kullanarak programın derlendiği anı



Şekil 17.1: Derleme Süreci

ve dosya adını ekrana basan bir "Sürüm Bilgisi" fonksiyonu yazınız.

- ◇ **Çöz:** Pi sayısını tanımlayan bir `#define PI 3.14159` makrosu ile verilen yarıçapa göre daire alanını hesaplayan bir `#define ALAN(r)` parametrelili makro yazınız.
- ◇ **Çöz:** `#include <dosya.h>` ile `#include "dosya.h"` arasındaki dosya arama sırası farkını açıklayınız.
- ◇ **Çöz:** `#define KARE(x) x * x` makrosunda `KARE(2 + 3)` çağırısı yapıldığında beklenen 25 yerine neden 11 sonucu elde edilir? İşlem önceliği hatasını engellemek için makro nasıl düzeltilmelidir?
- ◇ **Çöz:** `#ifdef`, `#ifndef`, `#else`, `#endif` yönergelerini açıklayınız. Kodun hata ayıklama (`debug`) modunda ekstra log basmasını sağlayan bir koşullu derleme yapısı yazınız.
- ◇ **Çöz:** Bir başlık dosyasının (`.h`) proje genelinde birden fazla kez eklenerek "Redefinition Error" vermesini önleyen `#ifndef` / `#define` / `#endif` yapısını (Include Guard) yazınız.
- ◇ **Çöz:** Ön işlemciye özel `#` (`stringification`) ve `##` (`token pasting`) operatörlerinin ne işe yaradığını açıklayan kod örnekleri yazınız.
- ◇ **Çöz:** `do { ... } while(0)` kalıbının çok satırlı makrolar yazılırken `if-else` bloklarında oluşturabileceği mantıksal hataları engellemek için neden bir standart teknik olarak kullanıldığını açıklayınız.
- ◇ **Çöz:** Hatanın oluştuğu dosya adını, satır numarasını ve verilen hata mesajını biçimlendirilmiş olarak basan gelişmiş bir `LOG_ERROR(msg)` parametrelili makrosunu tasarlayınız.